USER INTERFACE AND DESIGN

ROLL NO: 240701222 NAME: Kaaviya Sri PS

EXPERIMENT 2:

COMPARISON OF UI DESIGN BETWEEN TWO FIGMA INTERFACES

AIM: The aim is to develop and compare Command Line Interface (CLI), Graphical User Interface (GUI), and Voice User Interface (VUI) for the same task, and assess user satisfaction using Python (with Tkinter for GUI and Speech Recognition for VUI) and Terminal.

I. (CLI) Command Line Interface

```
error="Invalid, please input valid values\n"
log=dict()
wallet=0
latest=0
expense=["food","clothing","rent","grocery","subscription","bills","misc"]
income=["salary","gift","profit","refund","misc"]

def add_expense():
    global wallet
    global latest
    while True:
              m=input("\nEnter  expense  description  (food,  clothing,  rent,  grocery,
subscription, bills, misc): ")
        n=float(input("Enter amount spent: "))
        if m.lower() not in expense or n<0:
            print(error)
        else:
            wallet-=n
            log[latest+1]=["expense",m,'-'+str(n),wallet]
            latest+=1
            current()
            break


def add_income():
    global wallet
    global latest
    while True:
        m=input("\nEnter income type (salary, gift, profit, refund, misc): ")
        n=float(input("Enter amount recieved: "))
        if m.lower() not in income or n<0:
            print(error)
        else:
            wallet+=n
```

```
            log[latest+1]=["income",m,'+'+str(n),wallet]
            latest+=1
            current()
            break



def show_log():
    print("\nLOG OF WALLET:")
    print("\nSno\tIncome/Expense\tType\tAmount\tRemainder")
    for i in log:
        print(f"{i}\t{log[i][0]}\t\t{log[i][1]}\t{log[i][2]}\t{log[i][3]}")
    current()

def current():
    print("\nCurrent balance remaining:",wallet)

def menu():
    while True:
        print("\n\n1 - Show balance")
        print("2 - Show change log")
        print("3 - Add expense")
        print("4 - Add income")
        print("5 - Exit")
        ch=int(input("Enter which task you'd like to do: "))
        if ch==1:
            current()
        elif ch==2:
            show_log()
        elif ch==3:
            add_expense()
        elif ch==4:
            add_income()
        elif ch==5:
            print("Thank you! Spend wisely!")
            break
        else:
            print(error)

print("Welcome to Your Wallet Companion!")
menu()
```

SAMPLE OUTPUT:

```
Welcome to Your Wallet Companion!
1 - Show balance
2 - Show change log
3 - Add expense
4 - Add income
5 - Exit
Enter which task you'd like to do: 3

Enter expense description (food, clothing, rent, grocery, subscription, bills, misc): food
Enter amount spent: 360

Current balance remaining: -360.0

1 - Show balance
2 - Show change log
3 - Add expense
4 - Add income
5 - Exit
Enter which task you'd like to do: 4

Enter income type (salary, gift, profit, refund, misc): refund
Enter amount recieved: 100

Current balance remaining: -260.0

1 - Show balance
2 - Show change log
3 - Add expense
4 - Add income
5 - Exit
Enter which task you'd like to do: 2

LOG OF WALLET:

Sno     Income/Expense  Type    Amount  Remainder
1       expense         food    -360.0  -360.0
2       income          refund  +100.0  -260.0

Current balance remaining: -260.0

1 - Show balance
2 - Show change log
3 - Add expense
4 - Add income
5 - Exit
Enter which task you'd like to do: 5
Thank you! Spend wisely!
```

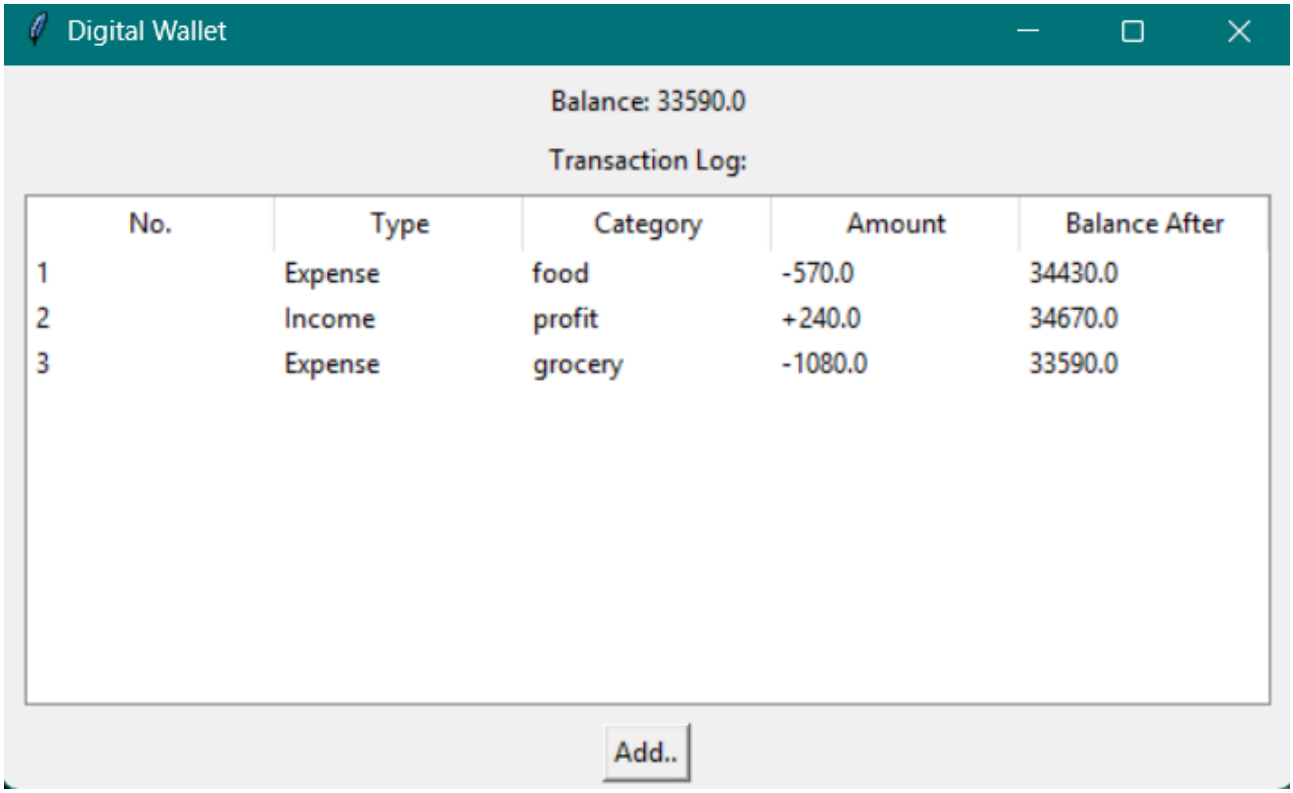
II. (GUI) Graphical User Interface

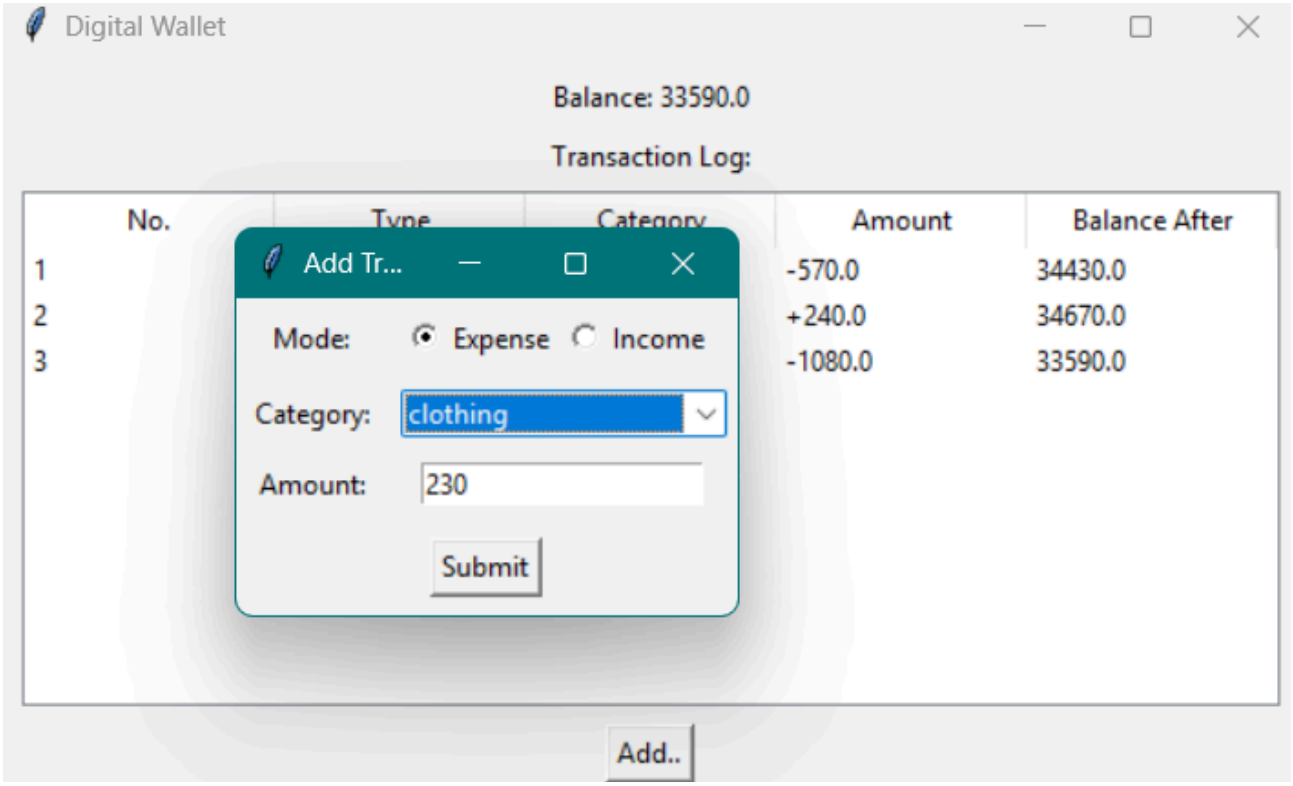
```
import tkinter as tk
from tkinter import ttk, messagebox

wallet = 35000.0
log = {}
latest = 0
expense_types = ["food", "clothing", "rent", "grocery", "subscription", "bills",
"misc"]
income_types = ["salary", "gift", "profit", "refund", "misc"]

root = tk.Tk()
root.title("Digital Wallet")

balance_var = tk.StringVar(value=f"Balance: {wallet}")
tk.Label(root, textvariable=balance_var).pack(pady=5)

tk.Label(root, text="Transaction Log:").pack()
cols = ("No.", "Type", "Category", "Amount", "Balance After")
tree = ttk.Treeview(root, columns=cols, show="headings", height=10)
for col in cols:
    tree.heading(col, text=col)
    tree.column(col, width=110)
tree.pack(padx=10, pady=5)

def refresh_balance():
    balance_var.set(f"Balance: {wallet}")

def refresh_log():
    for row in tree.get_children():
        tree.delete(row)
    for i, entry in log.items():
        tree.insert("", "end", values=(i, entry[0], entry[1], entry[2], entry[3]))

def open_add():
    win = tk.Toplevel(root)
    win.title("Add Transaction")

    mode_var = tk.StringVar(value="Expense")
    tk.Label(win, text="Mode:").grid(row=0, column=0, padx=5, pady=5)
    toggle_frame = tk.Frame(win)
    toggle_frame.grid(row=0, column=1, padx=5, pady=5, sticky="w")
     tk.Radiobutton(toggle_frame, text="Expense", variable=mode_var, value="Expense",
command=lambda: update_types()).pack(side="left")
```

```
     tk.Radiobutton(toggle_frame, text="Income", variable=mode_var, value="Income",
command=lambda: update_types()).pack(side="left")

    tk.Label(win, text="Category:").grid(row=1, column=0, padx=5, pady=5)
    type_var = tk.StringVar(value=expense_types[0])
          combo = ttk.Combobox(win, textvariable=type_var, values=expense_types,
state="readonly")
    combo.grid(row=1, column=1, padx=5, pady=5)

    def update_types():
        if mode_var.get() == "Expense":
            combo["values"] = expense_types
            type_var.set(expense_types[0])
        else:
            combo["values"] = income_types
            type_var.set(income_types[0])

    tk.Label(win, text="Amount:").grid(row=2, column=0, padx=5, pady=5)
    amt_entry = tk.Entry(win)
    amt_entry.grid(row=2, column=1, padx=5, pady=5)

    def submit():
        global wallet, latest
        try:
            n = float(amt_entry.get())
        except ValueError:
            messagebox.showerror("Error", "Enter a valid number.")
            return
        if n < 0:
            messagebox.showerror("Error", "Amount can't be negative.")
            return
        if mode_var.get() == "Expense":
            wallet -= n
            log[latest + 1] = ["Expense", type_var.get(), f"-{n}", wallet]
        else:
            wallet += n
            log[latest + 1] = ["Income", type_var.get(), f"+{n}", wallet]
        latest += 1
        refresh_balance()
        refresh_log()
        win.destroy()

    tk.Button(win, text="Submit", command=submit).grid(row=3, column=0, columnspan=2,
pady=8)
```

```
tk.Button(root, text="Add..", command=open_add).pack(pady=3)

root.mainloop()
```

SAMPLE OUTPUT:





III. (VUI) Voice User Interface

```
import speech_recognition as sr
import pyttsx3

wallet = 35000.0
log = {}
latest = 0
expense_types  =  ["food",  "clothing",  "rent",  "grocery",  "subscription",  "bills",
"misc"]
income_types  = ["salary", "gift", "profit", "refund", "misc"]

engine = pyttsx3.init()
recognizer = sr.Recognizer()

def speak(text):
    print(f"[Assistant]: {text}")
    engine.say(text)
    engine.runAndWait()

def listen(prompt):
    speak(prompt)
    with sr.Microphone() as source:
        recognizer.adjust_for_ambient_noise(source, duration=0.5)
        print("[Listening...]")
        try:
            audio = recognizer.listen(source, timeout=6, phrase_time_limit=6)
            result = recognizer.recognize_google(audio).lower().strip()
            print(f"[You said]: {result}")
            return result
        except sr.WaitTimeoutError:
            speak("I didn't hear anything. Please try again.")
            return None
        except sr.UnknownValueError:
            speak("Sorry, I couldn't understand that. Please try again.")
            return None
        except sr.RequestError:
            speak("Speech service is unavailable. Check your internet connection.")
            return None

def show_balance():
    speak(f"Your current balance is {wallet:.2f} rupees.")

def show_log():
    if not log:
```

```
        speak("No transactions recorded yet.")
        return
    speak(f"You have {len(log)} transactions.")
    for i, entry in log.items():
          speak(f"Transaction {i}: {entry[0]}, {entry[1]}, amount {entry[2]}, balance
after {entry[3]:.2f}.")

def add_expense():
    global wallet, latest    while True:
                    said  =  listen(f"What  kind  of  expense?  Say  one  of:  {',
'.join(expense_types)}.")
        if said is None:
            continue
        match = next((t for t in expense_types if t in said), None)
        if match:
            break
               speak(f"I  didn't  catch  a  valid  category.  Please  say  one  of:  {',
'.join(expense_types)}.")

    while True:
        said = listen("How much did you spend?")
        if said is None:
            continue
        words = said.replace("rupees", "").replace("rupee", "").strip().split()
        try:
            amount = float(words[0])
            if amount < 0:
                raise ValueError
            break
        except (ValueError, IndexError):
            speak("Please say a valid positive number.")

    wallet -= amount
    latest += 1
    log[latest] = ["Expense", match, f"-{amount}", wallet]
       speak(f"Got  it.  {amount:.2f}  rupees  spent  on  {match}.  Your  balance  is  now
{wallet:.2f} rupees.")

def add_income():
    global wallet, latest

    while True:
                  said  =  listen(f"What  is  the  income  source?  Say  one  of:  {',
'.join(income_types)}.")
```

```
        if said is None:
            continue
        match = next((t for t in income_types if t in said), None)
        if match:
            break
                speak(f"I  didn't  catch  a  valid  source.  Please  say  one  of:  {',
'.join(income_types)}.")

    while True:
        said = listen("How much did you receive?")
        if said is None:
            continue
        words = said.replace("rupees", "").replace("rupee", "").strip().split()
        try:
            amount = float(words[0])
            if amount < 0:
                raise ValueError
            break
        except (ValueError, IndexError):
            speak("Please say a valid positive number.")

    wallet += amount
    latest += 1
    log[latest] = ["Income", match, f"+{amount}", wallet]
      speak(f"Got  it.  {amount:.2f}  rupees  added  from  {match}.  Your  balance  is  now
{wallet:.2f} rupees.")

def menu():
    speak("Welcome to your Wallet Companion.")
    while True:
        said = listen(
            "What would you like to do? "
            "Say: balance, log, add expense, add income, or exit."
        )
        if said is None:
            continue

        if "balance" in said:
            show_balance()
        elif "log" in said or "history" in said:
            show_log()
        elif "expense" in said:
            add_expense()
        elif "income" in said:
```

```
            add_income()
        elif "exit" in said or "quit" in said or "stop" in said:
            speak("Thank you! Spend wisely!")
            break
        else:
            speak("Sorry, I didn't catch a valid command. Please try again.")

menu()
```

SAMPLE OUTPUT:

```
(Voice) Welcome to your wallet companion.
(Voice) What would you like to do? Say balance, log, add expense, add income, or exit.
(User) add expense
(Voice) What kind of expense? Say one of food, clothing, rent, grocery, subscription, bills, misc.
(User) subscription
(Voice) How much did you spend?
(User) 299
(Voice) Got it. 299 rupees spent on subscription. Your balance is now 34701.00 rupees.
(Voice) What would you like to do? Say balance, log, add expense, add income, or exit.
(User) log
(Voice) You have 1 transactions.
(Voice) Transaction 1: expense, subscription, amount -199.00, balance after 34701.00
(Voice) What would you like to do? Say balance, log, add expense, add income, or exit.
(User) exit
(Voice) Thank you! Spend wisely!
```